

NLP

- Explain TDF-IDF, LDA, Bag of Words.

LLMs / GenAI / Agents

- MoE (Mixture of Experts): Understanding how models like GPT-4 or Mixtral activate only specific "expert" sub-networks to save on inference costs.

Fine-Tuning (PEFT/LoRA): Knowing the math of "Low-Rank Adaptation"-how to update a tiny fraction of weights to specialize a model without retraining the whole thing.

-Knowledge Graphs: Moving beyond Vector DBs to graph-based relationships for complex

RAG.

KV Caching: Understanding how the model "remembers" previous tokens in a session to save compute power and reduce latency.

Guardrails & Safety: Implementing NeMo or LlamaGuard (Understanding the "Alignment" problem).

LLM-as-a-Judge (G-Eval): Building a deterministic rubric for an LLM to grade another LLM.

LLM-as-a-Judge is an evaluation method where a capable Large Language Model (e.g., GPT-4) assesses the outputs of another LLM application for quality, accuracy, and safety

Agentic Frameworks & Patterns: Mastery of orchestration patterns like Evaluator-Optimizer, Routing, and Parallelization.

- Advanced RAG: Deep knowledge of ReAG (Reasoning Augmented Generation) and how to improve retrieval using Knowledge Graphs. reason over full, unstructured documents rather than relying solely on pre-chunked semantic search
 - Transcript Evaluation: Analyzing the path the agent took (did it waste tokens? did it get stuck in a loop?).
 - Outcome Evaluation: Measuring if the agent actually solved the business problem (e.g., "Was the flight actually booked?").
 - Trajectory Success Rate: If an agent uses 5 tools to solve a task, did it take the most efficient path (Shortest Path) or did it "wander" through unnecessary tool calls?
- Agentic AI vs. AI Agents: Can distinguish between building a single autonomous script (Agent) and building a system where AI is embedded into the whole business process (Agentic AI).
- Agent frameworks: LangChain/LangGraph, AutoGen, CrewAI, among others.

Vector storage Indexing, search

Optimization

- Solving speed on a MIP (Mixed-Integer Programming) solver
- If you want to speed up solving time, what do you change? Objective function,
- variables, constraints?
 - What is the trade-off of heuristics on optimization models?

TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF (Term Frequency-Inverse Document Frequency) is a numerical statistic that measures the importance of a word to a document within a collection (corpus). It highlights unique, relevant terms by increasing the weight of words frequent in a specific document while lowering the weight of words that appear frequently across all documents. [Wikipedia +2](#)

How TF-IDF Works

It is calculated by multiplying two metrics:

- **Term Frequency (TF):** Measures how often a word appears in a specific document:

$$\frac{\text{Number of times word appears in document}}{\text{Total words in document}}$$

- **Inverse Document Frequency (IDF):** Measures how important a word is across the entire corpus, reducing the weight of common words (e.g., "the", "and"). It is

computed as the logarithm of $\frac{\text{Total number of documents}}{\text{Number of documents containing the word}}$. [EPFL +2](#)

Applications

- **Information Retrieval/Search Engines:** Ranking documents based on relevance to a query.
- **Keyword Extraction:** Identifying the most relevant keywords in a text.
- **Text Classification/Clustering:** Grouping similar documents based on content, such as identifying spam or classifying topics. [GeeksforGeeks +2](#)

What is Text Mining?

The technique of reviewing vast collections of documents in order to find new information or answer is known as text mining. Text mining uncovers facts, connections, and statements that would otherwise be lost in a sea of textual large data. After being extracted, the data is turned into a structured format

that can be further examined or displayed in a variety of ways. To process the text, text mining uses a range of approaches, one of the most essential of which being "Natural Language Processing (NLP)".

Topic modeling

A topic model is a form of statistical model used in machine learning and natural language processing to find abstract "topics" that appear in a collection of documents.

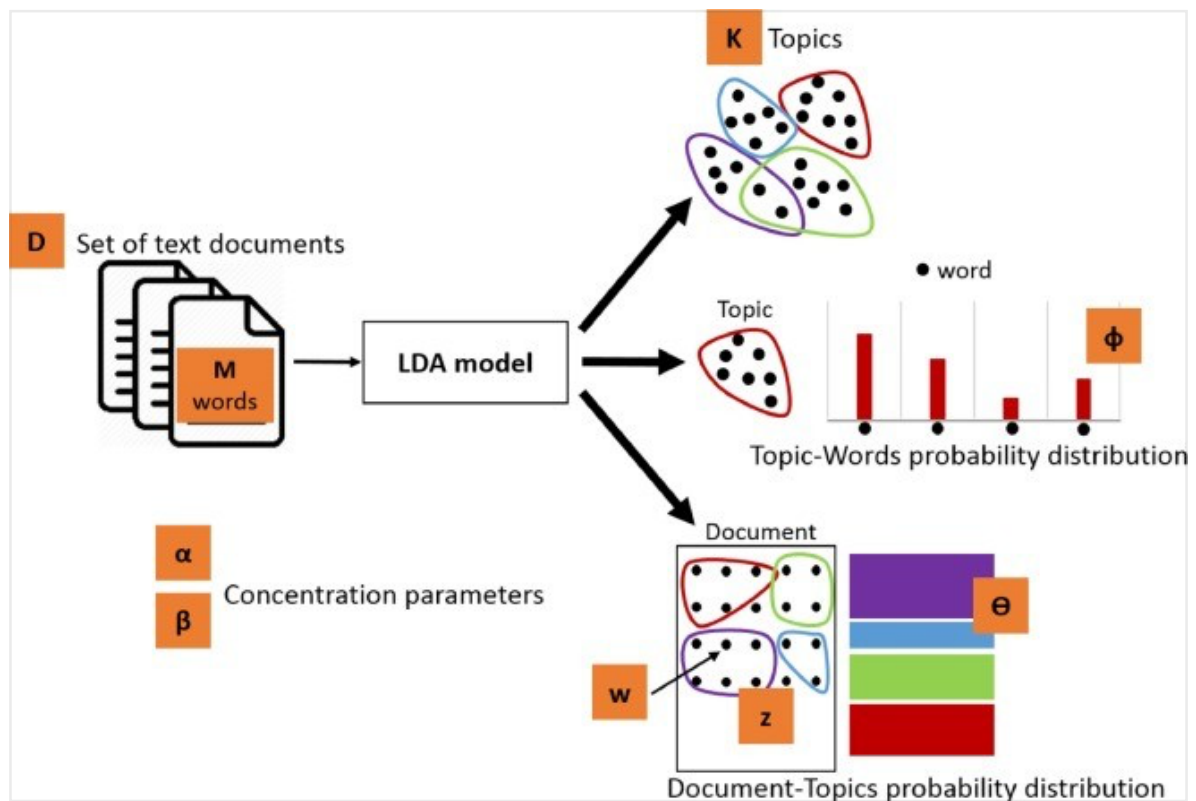
Topic Modeling is an unsupervised learning method for clustering documents and identifying topics based on their contents. It works in the same way as the K-Means algorithm and Expectation-Maximization. We will have to evaluate individual words in each document to uncover topics and assign values to each depending on the distribution of these terms because we are clustering texts.

Latent Dirichlet Allocation.

Latent Dirichlet Allocation (LDA) is used to classify text in a document to a certain topic. It creates a Dirichlet distribution based on topic per document and word per topic model.

How does LDA Work?

- 1) LDA assigns words at random to k topic for each document, where k is the number of pre-defined topics.
- 2) LDA computes for each document ' d ' and each word ' w ' in the text.
 - a. $P(\text{topic}(t) \mid \text{document}(d))$: Proportion of words allocated to subject t in document d .
 - b. $P(\text{word}(w) \mid \text{topic}(t))$: Proportion of topic t assignment over all documents derived from w .
- 3) Given all of the other words and their topic assignments, reassign topic t to word w with probability p .
- 4) Iterate multiple times until the topic assignment remains the same.



MoE — Mixture of Experts

Core Idea

Instead of running the **entire network** for every token, MoE splits the feed-forward layers into N **expert sub-networks** and only activates K of them per token. Same model capacity, fraction of the compute cost.

Standard FFN: every token goes through one big FFN

MoE FFN: every token goes through 2 of 8 experts (for example)

Active params per token $\ll$ Total params in model

Architecture

Token $\rightarrow$ Router (gating network) $\rightarrow$ selects Top-K experts

$\rightarrow$ Expert 1 (FFN) $\neg$

$\rightarrow$ Expert 3 (FFN) $\neg$ $\rightarrow$ weighted sum $\rightarrow$ output
(others ignored) $\neg$

Router score: $g_i = \text{softmax}(W_{\text{router}} \cdot x)$

Final output: $y = \sum g_i \cdot \text{Expert}_i(x)$ for top-K experts only

ELI5: A hospital where every patient goes to triage first. Triage decides: send this patient to cardiology + neurology, ignore all other departments. The hospital has 100 specialist departments but only 2 are activated per patient.

Why It Matters

GPT-4 (rumored): ~8 experts, 2 active per token

Mixtral 8x7B: 8 experts of 7B params each, 2 active

Total params: ~47B

Active params per token: ~13B ← inference cost of 13B model

You get the quality of a large model at the inference cost of a small one.

Load balancing problem: without extra regularization, the router collapses — sends everything to 1-2 experts, ignoring the rest. Fix: auxiliary loss that penalizes uneven expert usage.

Pros & Cons of MoE

Pros	Cons
Massive capacity at low inference cost	High memory — all experts must be loaded
Specialization — experts learn different skills	Training instability, routing collapse risk
Scales well with more experts	Communication overhead in distributed training
State of the art efficiency	Harder to fine-tune than dense models

Fine-Tuning — PEFT / LoRA

The Problem

Fine-tuning all weights of a 7B parameter model is expensive — requires storing gradients and optimizer states for 7 billion parameters. PEFT (Parameter-Efficient Fine-Tuning) updates only a tiny fraction of weights.

LoRA — Low-Rank Adaptation

Core math idea: weight updates during fine-tuning tend to have **low intrinsic rank** — they live in a small subspace. So instead of updating the full weight matrix W , approximate the update with two small matrices.

Original weight matrix: $W \in \mathbb{R}^{(d \times d)}$ (frozen, not updated)

LoRA adds a bypass: $\Delta W = A \cdot B$

$A \in \mathbb{R}^{(d \times r)}$ $B \in \mathbb{R}^{(r \times d)}$ $r \ll d$

Forward pass: $y = (W + A \cdot B) \cdot x$
 ↑ frozen ↑ trained

Parameter savings example:

W is $1024 \times 1024 = 1,048,576$ parameters (frozen)

$r = 8$:

A is $1024 \times 8 = 8,192$

B is $8 \times 1024 = 8,192$

Total trainable: 16,384 ← 64× fewer parameters

ELI5: Instead of repainting the entire wall, you put a thin sticker on top. The sticker is the low-rank update — tiny but enough to change the behavior.

LoRA Initialization

A initialized with random Gaussian → breaks symmetry

B initialized with zeros → $\Delta W = A \cdot B = 0$ at start
 model starts identical to base

This is critical — training starts from the pretrained model's behavior, not random noise.

Key Hyperparameters

r = rank of the update (4, 8, 16, 32 — higher r = more capacity)

α = scaling factor, ΔW is scaled by α/r
common to set $\alpha = r$ (or $2r$)

target modules = which layers to apply LoRA to
 typically: Q, V attention matrices
 sometimes: K, O, FFN layers too

LoRA Variants

QLoRA — LoRA + quantize the base model to 4-bit. Fine-tune a 65B model on a single GPU. The quantization reduces memory, LoRA reduces trainable parameters.

AdaLoRA — adaptively allocates rank r across layers. Important layers get higher rank, minor layers get lower rank.

DoRA — decomposes weights into magnitude + direction, applies LoRA to direction only. Often better than vanilla LoRA.

PEFT Methods Comparison

Method	Trainable Params	Approach	Best For
--------	------------------	----------	----------

Full fine-tune	100%	Update all weights	Lots of data, compute
LoRA	~0.1–1%	Low-rank weight update	General fine-tuning
QLoRA	~0.1–1%	LoRA + 4-bit quantization	Single GPU, large models
Prefix tuning	<1%	Learn soft prompt tokens	Few-shot, frozen model
Adapters	~1–5%	Small bottleneck layers	Multi-task learning

LLM Agents — Quick Overview

Agents extend LLMs beyond text generation by giving them **tools** and a **decision loop**:

Loop:

1. LLM receives task + context
2. LLM decides: think / use tool / respond
3. If tool → execute (search, code, API call)
4. Result added to context
5. Repeat until task complete

Key components:

- **Reasoning** — chain-of-thought, ReAct (reason + act interleaved)
- **Memory** — in-context (limited), vector DB (long-term retrieval)
- **Tools** — web search, code interpreter, APIs, databases
- **Planning** — task decomposition (ReAct, Tree of Thought, AutoGPT-style)

RAG (Retrieval-Augmented Generation) — instead of memorizing all facts, retrieve relevant documents at inference time and inject into context. Reduces hallucination, keeps knowledge fresh.

KV Caching:

Understanding how the model "remembers" previous tokens in a session to save compute power and reduce latency.

How Does KV Caching Work?

Step-by-Step Process

1. **First Generation:** When the model sees the first input, it calculates and stores its keys and values in the cache. ↴
2. **Next Words:** For each new word, the model retrieves the stored keys and values and adds the new ones instead of starting over.
3. **Efficient Attention Computation:** calculate attention using the

cached K
K and V
V along with the new Q
Q (query) to compute the output.

4. **Update Input:** add the newly generated token to the input and go back to step 2 go back to step 2 until we finish generating.

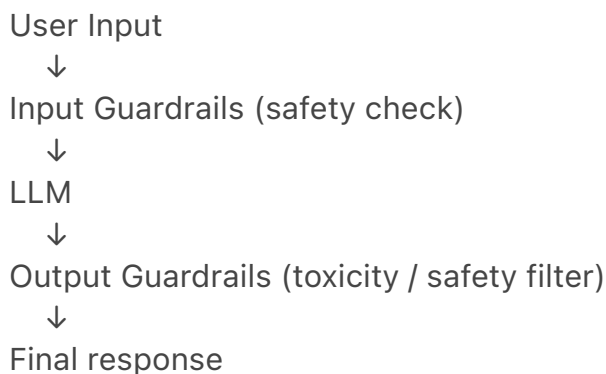
KV caching is a simple but powerful technique that helps AI models generate text faster and more efficiently. By remembering past calculations instead of repeating them, it reduces the time and effort needed to predict new words. While it does require extra memory, this method is especially useful for long conversations ensuring fast and efficient generation

Guardrails

Guardrails are systems that **monitor, filter, or correct model inputs and outputs** to enforce safety policies.

They act as **control layers around the LLM**.

Architecture



Guardrails Using LlamaGuard

LlamaGuard: <https://huggingface.co/meta-llama/Llama-Guard-3-8B>

LlamaGuard is a **safety classification model** developed to detect **unsafe prompts and responses**.

It classifies text according to **safety policies**.

Example Categories

Category	Description
violence	harmful actions

self-harm	suicide content
illegal activity	criminal guidance
harassment	abusive language

Example

Input prompt:

How can I break into a car?

Classifier output:

Unsafe: Criminal Activity

Then the system **blocks the request**

Concept	Meaning
Alignment	ensuring AI behavior follows human values
Guardrails	safety controls around LLMs
NeMo Guardrails	rule-based conversation safety framework
LlamaGuard	classifier detecting unsafe prompts/responses

Reasoning-Augmented Generation (ReAG)

is an emerging approach in AI that integrates a language model’s reasoning process directly into the content generation pipeline, especially for knowledge-intensive tasks. In a traditional **Retrieval-Augmented Generation (RAG)** setup, a query is answered in two stages: first, retrieving documents (often via semantic similarity search) and then generating an answer from those documents (ReAG: Reasoning-Augmented Generation — Superagent).

While effective, this RAG approach can fail to capture deeper contextual links — it may retrieve text that looks similar to the query but misses relevant information (ReAG: Reasoning-Augmented Generation — Superagent). ReAG was introduced to overcome these limitations by essentially skipping the separate retrieval step (ReAG: Reasoning-Augmented Generation — Superagent). Instead of relying on pre-indexed snippets or purely surface-level matches, ReAG feeds raw source materials (e.g., full-text files, web pages, or spreadsheets) directly into a large language model (LLM), allowing the model itself to determine what information is useful and why (ReAG: Reasoning-

Augmented Generation — Superagent).

RAG = Retrieve → Then Answer

Pipeline:

1. User asks question
2. System retrieves relevant documents/chunks
3. LLM answers using retrieved context

Goal: Ground answers in external knowledge.

Typical use case:

- FAQ bots
- Document Q&A
- Internal knowledge assistants

Source/background:

ReAG = Add Explicit Reasoning / Smarter Context Selection

Idea: Instead of blindly using retrieved chunks, the model/reasoning layer:

- evaluates which evidence is actually relevant,
- may reason over multiple pieces of evidence,
- may iteratively refine retrieval,
- may filter noisy context before answering.

Depending on the paper/framework, ReAG can also mean letting the model reason directly over raw docs instead of a basic retrieve-then-generate pipeline. The exact implementation varies by author/tool

Why Knowledge Graphs Help Retrieval

Vector search alone can struggle with:

- **Multi-hop questions**
("Which suppliers work with companies acquired by X?")
- **Ambiguous entities**
("Apple" company vs fruit)
- **Fragmented evidence across documents**
Relevant facts may live in separate chunks.
- **Needing relational context**
Some answers depend on how entities connect.

Main Ways KGs Improve Retrieval

1. Graph-Based Expansion

Retrieve an initial entity/chunk, then expand to connected nodes.

Example:

User asks:

"What papers influenced Geoffrey Hinton's work on transformers?"

Pipeline:

1. Retrieve node: Geoffrey Hinton
2. Traverse graph to related papers / collaborators / citations
3. Retrieve supporting documents for those nodes

Benefit: Better multi-hop retrieval.

Multi-Hop Reasoning — The Key Advantage

Vector DBs retrieve isolated chunks. Knowledge graphs can **chain relationships** across many steps:

Query: "What companies were founded by people who worked at PayPal?"

Multi-hop traversal:

PayPal → [employed] → Peter Thiel → [founded] → Palantir

PayPal → [employed] → Elon Musk → [founded] → Tesla, SpaceX

PayPal → [employed] → Reid Hoffman → [founded] → LinkedIn

A vector DB would need to get lucky finding all these in one chunk.

A knowledge graph answers this in one traversal.

GraphRAG — Combining Both

Standard RAG: embed documents → retrieve top-K chunks → feed to LLM.

GraphRAG (Microsoft, 2024): build a knowledge graph from documents first, then use both graph traversal AND vector similarity for retrieval.

Document ingestion:

Text → Entity extraction → Relation extraction → Knowledge Graph
+ Vector embeddings

Query time:

1. Embed query → find relevant entities (vector search)

2. Traverse graph from those entities (multi-hop)

3. Collect subgraph + relevant chunks

4. Feed enriched context to LLM

Why it matters: dramatically reduces hallucination on complex multi-entity questions. The LLM gets structured facts, not just similar-sounding text.

Agentic frameworks organize **LLMs + tools + workflows** so models can **reason, act, and coordinate tasks**. Instead of a single prompt, they use **orchestration patterns** that control how multiple steps interact. Below are the **three key orchestration patterns** you mentioned.

Evaluator–Optimizer Pattern

Idea

One model **generates outputs**, another model **evaluates them**, and the system **iteratively improves the result**.

This mimics **human editing or peer review**.

Workflow

User input



Generator (LLM produces answer)



Evaluator (LLM critiques output)



Optimizer revises answer



Repeat until good enough

Example

Task: write a research summary.

Steps:

1. LLM writes summary
2. Evaluator checks:
 - accuracy
 - clarity
 - completeness
3. Model revises based on feedback.

Example prompt flow:

Draft summary → Evaluate → Improve → Final summary

Routing Pattern

Idea

A **router model decides which tool, model, or workflow should handle the request**.

This is common in **multi-agent systems**.

Workflow

User query



Router LLM



Select specialized tool/agent



Execute task

Example

User asks:

"What is the derivative of x^2 ?"

Router decides:

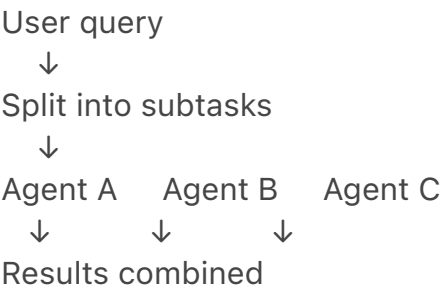
Query type	Route
math	math solver
coding	code agent
search	retrieval system

Parallelization Pattern

Idea

Multiple agents **run tasks simultaneously**, then results are **combined**.
This reduces latency and improves coverage.

Workflow



Example

Task: analyze a research paper.
Parallel tasks:

Agent	Task
Agent 1	summarize
Agent 2	extract key results
Agent 3	identify limitations

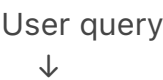
Final step merges outputs.

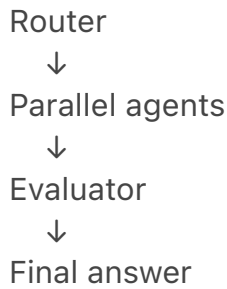
Comparison of the Patterns

Pattern	Main Goal	When to Use
Evaluator–Optimizer	improve output quality	writing, reasoning
Routing	choose best tool	multi-tool systems
Parallelization	reduce latency / expand analysis	complex tasks

Combined Architecture (Typical Agent System)

Real systems often combine all three:





This architecture is common in **modern AI assistants and autonomous agents**.

Example: Research Assistant Agent

Step-by-step system:

1. Router → classify question
2. Parallel agents → search papers, summarize, extract results
3. Evaluator → check consistency
4. Final answer generator

Example: Research Assistant Agent

Step-by-step system:

1. Router → classify question
2. Parallel agents → search papers, summarize, extract results
3. Evaluator → check consistency
4. Final answer generator

The Alignment Problem

The **alignment problem** in AI means ensuring that a model's behavior is **consistent with human values, safety policies, and intended goals**.

Large language models are trained to **predict the next token**, not necessarily to **behave safely or ethically**. Therefore they may produce:

- harmful instructions
- misinformation
- biased content
- unsafe outputs

So we must add **alignment mechanisms**.

Concept

Model Objective

≠

Human Intent

$\text{Model Objective} \neq \text{Human Intent}$

Model Objective $\neq$ Human Intent

Goal of alignment:

Model Behavior

≈

Human Values + Safety Constraints

$\text{Model Behavior} \approx \text{Human Values + Safety Constraints}$

Model Behavior ≈ Human Values + Safety Constraints

Interview Phrasing

"KV caching stores the Key and Value projections of all past tokens so the model doesn't recompute them at each generation step. It reduces time complexity from $O(n^2)$ to $O(n)$ per token, at the cost of memory that grows linearly with context length. GQA and PagedAttention are the main techniques to make this memory manageable at scale."

Letter	Name	Question
Q	Query	"What am I looking for?"
K	Key	"What do I contain / offer?"
V	Value	"What do I actually give if selected?"

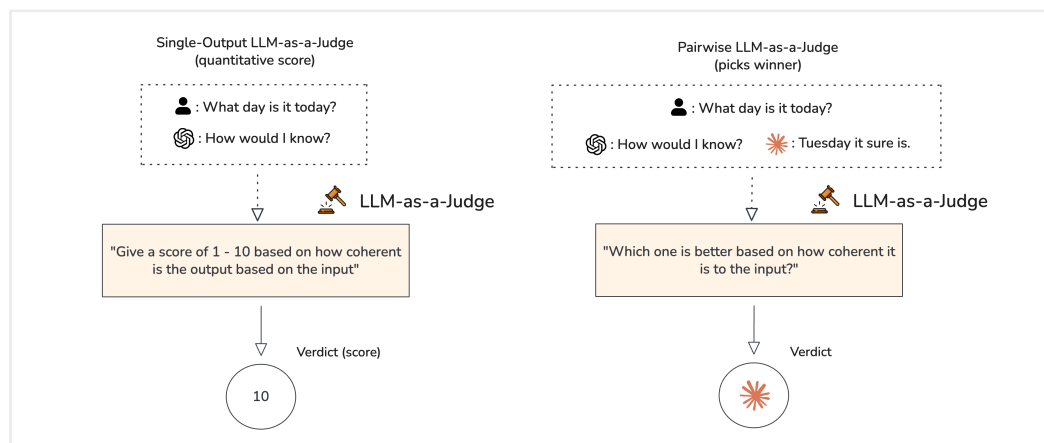
Pretraining = self-supervised (labels from data itself)

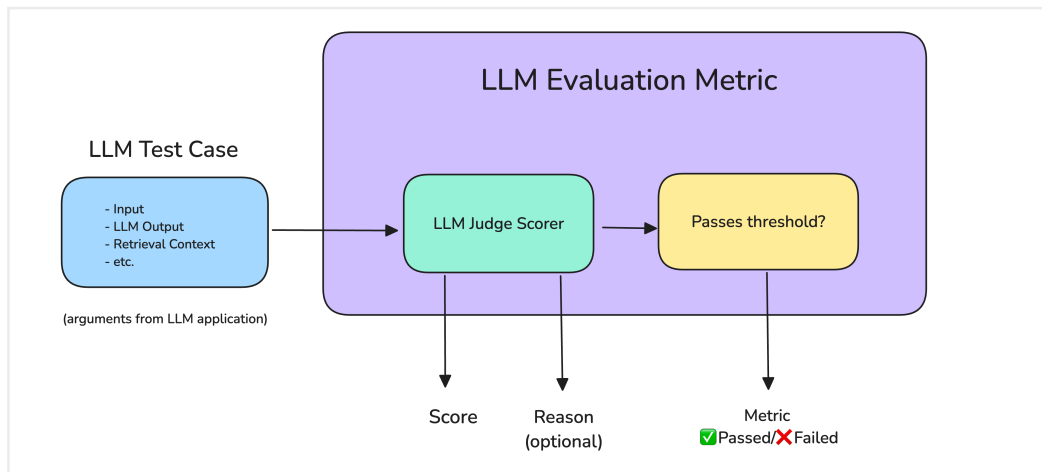
SFT = supervised (human written examples)

RLHF = reinforcement (reward signal from humans)

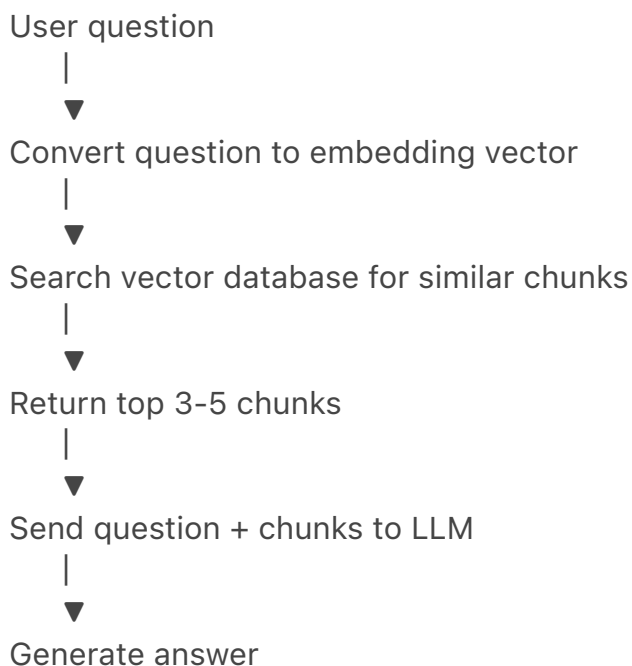
ELI5: Reading a book and answering questions. Without cache: reread the entire book before answering each question. With cache: read once, keep notes (K, V), just consult your notes for each new question.

PART 3 — LLM-as-a-Judge / G-Eval





Basic RAG



EVALuation Metrics

hallucination

is a confident response from an AI that is factually incorrect, nonsensical, or untruthful, often occurring because the model prioritizes plausible-sounding text over factual accuracy. An LLM hallucination is a confident response from an AI that is factually incorrect, nonsensical, or untruthful, often occurring because the model prioritizes plausible-sounding text over factual accuracy.

Summary ``` BASIC RAG: question → one search → retrieve → generate → simple, fast, cheap → fails on complex questions ReAG ADDS: 1. decomposition → break question into sub-questions 2. targeted search → one search per sub-question 3. evaluation → verify retrieval is sufficient 4. refinement → search again if gaps found 5. loop → repeat until confident KEY INSIGHT: The model reasons about WHAT it needs before and after retrieval not just at generation time



ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE is a metric used to evaluate **generated text** by comparing it to one or more **reference texts**.

It is especially common for:

- text summarization
- machine translation (less common now)
- general NLG evaluation

Reference: The cat sat on the mat

Generated: The cat is on mat

ROUGE-1 overlap:

Matched words:

The, cat, on, mat

Limitations of ROUGE

Major limitation:

It measures lexical overlap, not semantic quality.

Example:

Reference: The car is fast

Generated: The automobile is quick

ROUGE score may be low despite **same meaning**.

8. Modern Alternatives

Because of this limitation, newer metrics are often preferred:

Metric	Improvement
BLEU	Precision-based n-gram metric
BERTScore	Semantic similarity via embeddings

METEOR	Synonym-aware matching
LLM-as-Judge	Model-based evaluation

Reference (human)

R: "The cat is sitting on the mat."

Candidate (model output)

C: "A cat sits on the rug."

Semantically: **perfectly fine translation/paraphrase.**

Lexically: **very different words.**



BLEU (n-gram overlap)

BLEU only counts **exact word matches**.

Unigrams in common:

cat, on, the

Words that do **not** match:

- *sitting* ≠ *sits*
- *mat* ≠ *rug*
- *The* ≠ *A*

Bigrams in common: almost none.

So precisions are very low:

- $P_1 \approx 3/6$
 $p_1 \approx 3/6$
- $p_2 \approx 0$
 $p_2 \approx 0$
- $p_3 \approx 0$
- $p_4 \approx 0$

BLEU ≈ 0

BLEU ≈ 0

👉 BLEU concludes: **bad translation**

👉 Human concludes: **perfectly fine**



BERTScore (semantic token similarity)

BERTScore embeds words using **BERT** and compares cosine similarity.

What it "sees" in embedding space:

Candidate word	Best match in reference	Cosine similarity (conceptual)
cat	cat	1.00
sits	sitting	0.92
rug	mat	0.89

on	on	1.00
a	the	0.75

Average similarity $\approx$ **0.91**

BERTScore

$\approx$

0.9

$\text{BERTScore} \approx 0.9$

$\text{BERTScore} \approx 0.9$

COMET (learned human judgment)

COMET also sees the **source sentence** (important).

Assume source (Spanish):

"El gato está sentado en la alfombra."

COMET processes:

- source
- candidate
- reference

It has been trained on human ratings and has learned that:

- *rug* $\approx$ *mat*
- *sits* $\approx$ *is sitting*

COMET outputs something like:

COMET

=

0.96

$\text{COMET} = 0.96$

$\text{COMET} = 0.96$

 COMET concludes: **excellent translation (human-level)**

Metric	What it notices	Verdict (example)	Advantages	Disadvantages	Common alternatives
BLEU	Exact n-gram word overlap	 Bad	Very fast; no model needed; language-agnostic; historic MT benchmark	Fails on paraphrase/synonyms; ignores meaning; poor for LLM outputs, summarization, QA	ROUGE, BERTScore

ROUGE	Overlap of n-grams / longest common subsequence (recall-oriented)	 Bad	Good for summarization; simple; fast; recall focus (did we capture key words?)	Still lexical; misses paraphrase; can reward verbosity	BERTScore, MoverScore
BERTScore	Semantic similarity via embeddings	 Very good	Handles synonyms / paraphrase; strong human correlation; works across tasks (MT, QA, summarization)	Slower; needs a model; token-level (limited reasoning)	MoverScore, Sentence-Transformers cosine
COMET	Learned human judgment using source + ref	 Excellent	State-of-the-art for MT; uses source context; best human correlation	Heavy model; mainly MT-focused; slower	BLEURT, UniTE

Method	Needs reference?	Understands meaning?	Human-aligned?	Best use
BLEU				Legacy MT
ROUGE				Summarization baseline
BERTScore			 medium	semantic similarity
COMET	 (often)		 high	MT evaluation

G-Eval	✗	✓	🟢 very high	general LLM eval
LLM-as-a-Judge	✗	✓	🟢 very high	production LLM evaluation

G EvAL

G-Eval is a framework that uses Large Language Models (LLMs) like GPT-4, combined with Chain-of-Thought (CoT) prompting, to evaluate the quality of Natural Language Generation (NLG) outputs based on custom criteria.

Task: Evaluate a summary

Input text (source):

"The company launched a new AI model that improves translation quality and reduces cost by 30%."

Model output (summary):

"The company released an AI system that makes translation cheaper and better."



G-Eval prompt structure (what the LLM sees)

You are an expert evaluator.

Evaluate the summary based on:

1. Faithfulness (is it factually correct?)
2. Clarity (is it easy to understand?)
3. Conciseness (is it brief?)

Think step by step and give scores from 1–5.



LLM-as-a-Judge (example)

It:

- compares outputs directly (A vs B)
- uses reasoning like a human evaluator
- outputs preference or ranking
-

Task: Compare two answers

Question:

"Why is machine learning useful?"

Answer A

"Machine learning helps computers learn patterns from data and improve automatically."

Answer B

"Machine learning is when computers are programmed to follow rules written by humans."

Judge prompt

You are an expert evaluator.

Which answer is better and why? Consider correctness and completeness.

Evaluation:

- Answer A is correct and complete.
- Answer B is incorrect (ML is not rule-based programming).

Decision:

👉 Answer A is better



Perplexity

is a fundamental evaluation metric in natural language processing (NLP) that measures how well a probability model, specifically a language model (LM), predicts a sample.

Frameworks :

Here's the plain text table — copy-paste ready:

Framework	Mental Model	RAG Support	Chunking	Embeddings	Vector DBs	Knowledge Graphs	Ontologies	Strengths	Weaknesses	Best For	Avoid When	Hybrid Arch
-----------	--------------	-------------	----------	------------	------------	------------------	------------	-----------	------------	----------	------------	-------------

Lang Chain	Pipeline / Chain	★ ★ ★ Full stack	Via text splitters	20+ providers	50+ integrations	Partial — Neo4j plugin	No native	Huge ecosystem, fast prototyping, most integrations	Abstraction leaks, hard to debug, heavy deps	Prototyping, standard RAG, batteries-included teams	Need full control, low latency, minimal deps	Pairs with LangGraph for agents
Lang Graph	Stateful Graph / DAG	★ ★ Via LangChain	Via LangChain	Via LangChain	Via LangChain	Partial — flow graph only, not data	No	Complex agents, branching logic, human-in-the-loop, persistence	Step learning curve, verbose, overkill for linear flows	Multi-agent systems, approvals, flows, long-running workflows	Simple Q&A, one-shot pipelines, LLM beginners	Orchestrates LangChain + LlamaIndex tools

Llama Index	Index / Retrieval Engine	★★★★ Best-in-class	Advanced — semantic, sentence, hierarchical	20+ providers	40+ integrations	Strong — native KG Index	Partial — entity/relation extraction	RA G quality, query routing, hybrid search, re-ranking	Smaller agent ecosystem, less mature UI tooling	Production RA G, document Q&A, structured + unstructured data	Pur age nt orches tration	Bes t RA G lay er for any sta ck
Google ADK	Agent / Tool-use	★★★ Basic	Via Vertex AI	Via Vertex AI	Via Vertex AI Search	Partial — Google KG API	No	Native Gemini, GCP services, enterprise auth	GCP lock-in, weak RA G primitives, immature outside GCP	Google Cloud shops, Gemini-first, enterprise GCP	Mul ti-clo ud, adv anc ed RA G, non-G CP infr a	Plu gs into GC P dat a ser vic es

Cre wAI	Rol e- bas ed Mul ti- age nt	★ Min ima l	No nati ve	Via tool s	Via tool s	No ne	No	Int uiti ve role / tas k mo del, gre at DX, eas y mul ti- age nt	No RA G, no gra ph sup por t, limi ted pro d rea din ess	Tas k del ega tion , sim ulat ing tea m coll abo rati on	Pro duc tion RA G, co mpl ex stat e, ent erp rise de plo ym ent s	Wr ap Lla mal nde x for retr iev al tool s
Ha yst ack	Pip elin e / Co mp one nt Gra ph	★ ★ ★ Str ong	Mo dul ar pre pro ces sor s	15+ pro vid ers	30 + inte gra tion s	Par tial — cus tom co mp one nts	No	Pro duc tion -gr ade , mo dul ar, typ e- saf e, NL P- hea vy app s	Sm alle r co mm unit y, les s age nt tool ing, ste epe r tha n Lan gC hai n	Ent erp rise sea rch , doc um ent pro ces sin g, on- pre m	Qui ck pro tot yping, hea vy age nt nee ds	Go od sta nda lon e or wit h Lan gGr aph

Neo4j es una base de datos orientada a grafos, específicamente clasificada como un sistema NoSQL de "propiedad de grafos" (property graph). +

Framework	Purpose	When to Use
LangChain	Modular framework for building LLM-powered applications by chaining prompts, tools, memory, retrievers, and external integrations.	Use when building standard LLM pipelines, tool-calling workflows, RAG systems, or prototypes requiring broad ecosystem support.
LangGraph	Graph-based orchestration framework for designing stateful, multi-step, branching, and multi-agent workflows on top of LangChain.	Use when workflows require loops, branching logic, persistent state, human-in-the-loop steps, or complex agent orchestration.
LlamaIndex	Data framework specialized for connecting LLMs with external/private data sources through indexing, retrieval, and RAG pipelines.	Use when the primary challenge is retrieval over documents/data rather than agent orchestration; ideal for advanced RAG systems.
Google ADK	Framework for building production-grade AI agents with structured tool use, orchestration, evaluation, and deployment support in Google's ecosystem.	Use when building enterprise or production agents, especially if leveraging Google infrastructure/tools or requiring structured agent deployment patterns.
CrewAI	Multi-agent framework focused on role-based autonomous agent collaboration and task delegation between specialized agents.	Use when modeling collaborative multi-agent systems where agents have distinct roles, responsibilities, and delegated subtasks.

Haystack	NLP/LLM orchestration framework focused on production search, retrieval, QA pipelines, and scalable RAG architectures.	Use when building production retrieval/search systems, document QA, or scalable enterprise-grade RAG pipelines.
----------	--	---

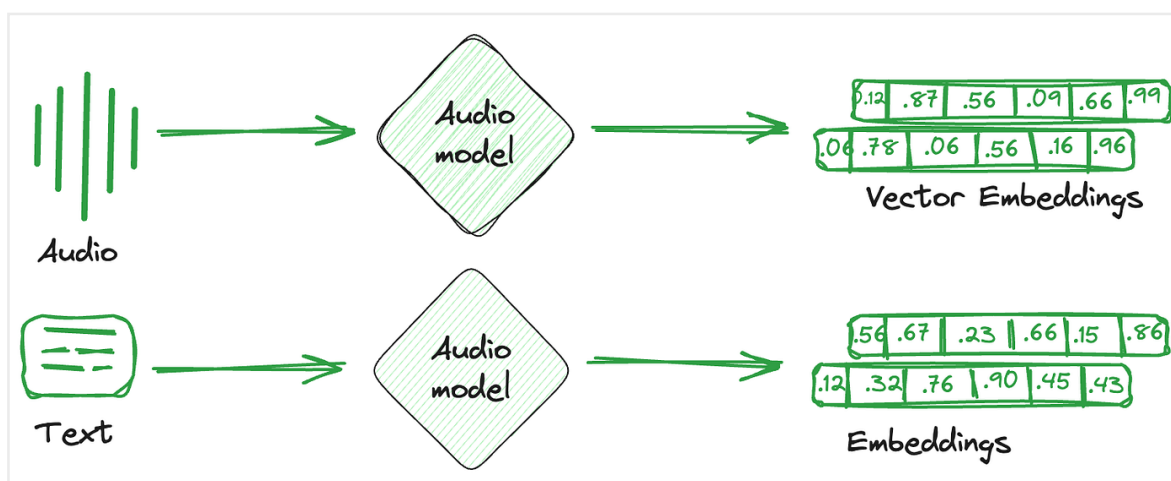
Vector storage

At a foundational level, vector databases store embeddings. Each has a fixed number of dimensions and is typically stored alongside metadata such as title, source, timestamp or category, which can be queried using metadata filters.

- [Chroma](#)^[7]
- [MongoDB](#)

Vector indexing

To accelerate similarity search in high-dimensional space, vector databases create indexes on stored vector embeddings. Indexing maps the vectors to new data structures, enabling faster similarity or distance searches between vectors.



Inverted File (IVF)

This is the most basic indexing technique. It splits the whole data into several clusters using techniques like K-means clustering. Each vector of the database is assigned to a specific cluster.

Navigable Small World (NSW)

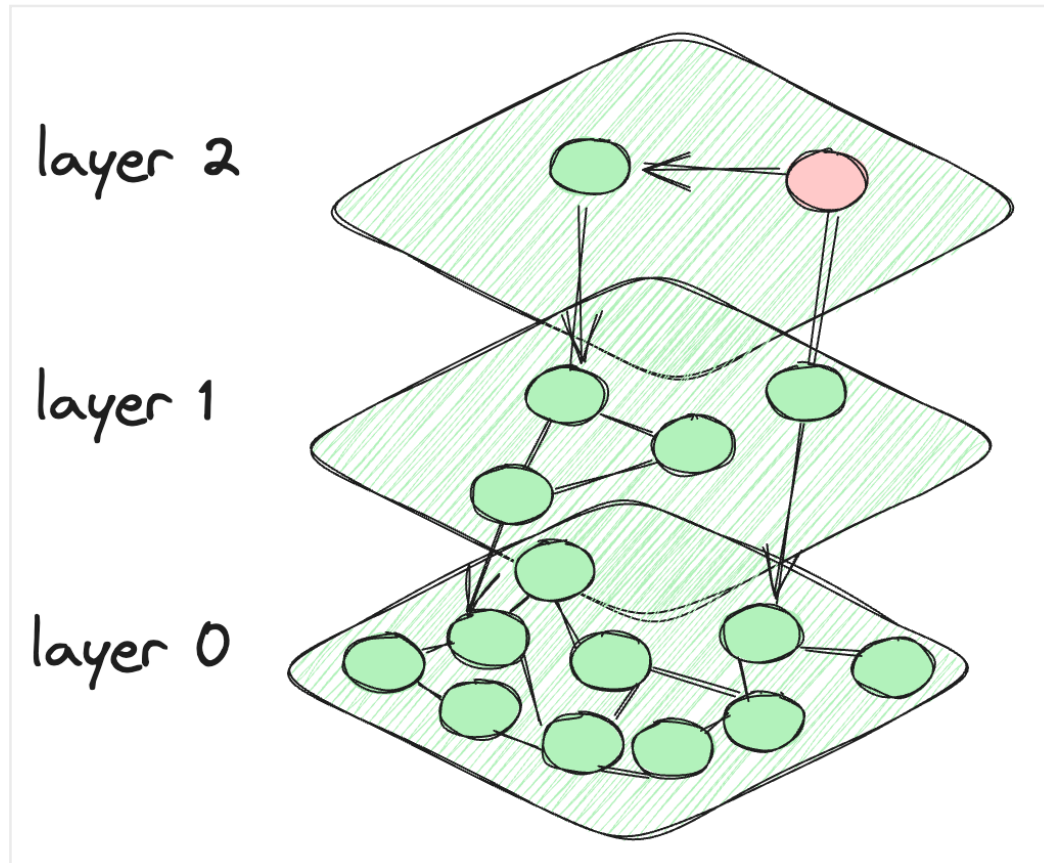
Navigable Small World (NSW) is similar to a proximate graph where nodes are linked together based on how similar they are to each other. The greedy method is used to search for the nearest neighbor point.

How HNSW (Hierarchical navigable small world) is Developed

So, what happens in HNSW is that we take the motivation from the skip list, and it creates layers like the skip list. But for the connection between the data

points, it makes a graph-like connection between the nodes. The nodes at each layer are connected not only to the current layer nodes but also to the nodes of the lower layers. The nodes at the top are very few and intensity increases when we go down to the lower layers. The last layer contains all the data points of the database. This is what the HNSW architecture looks like.

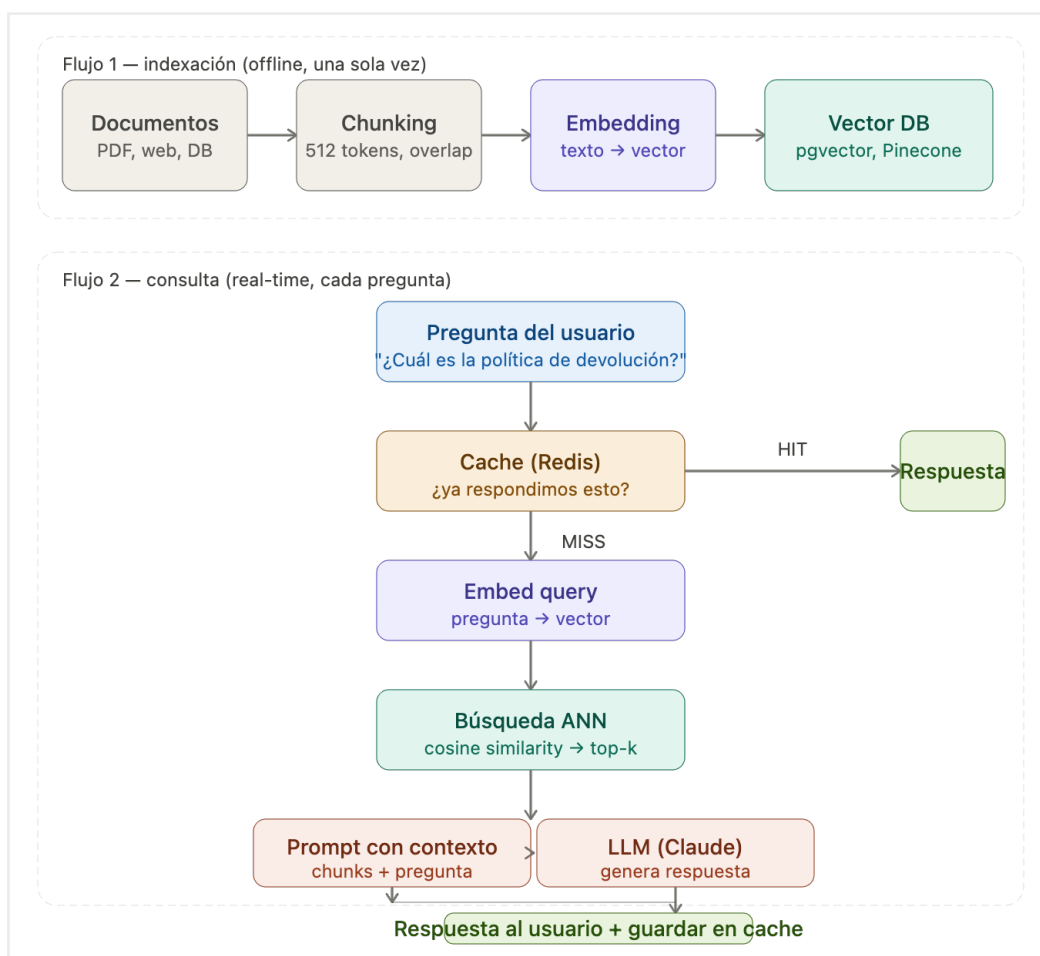
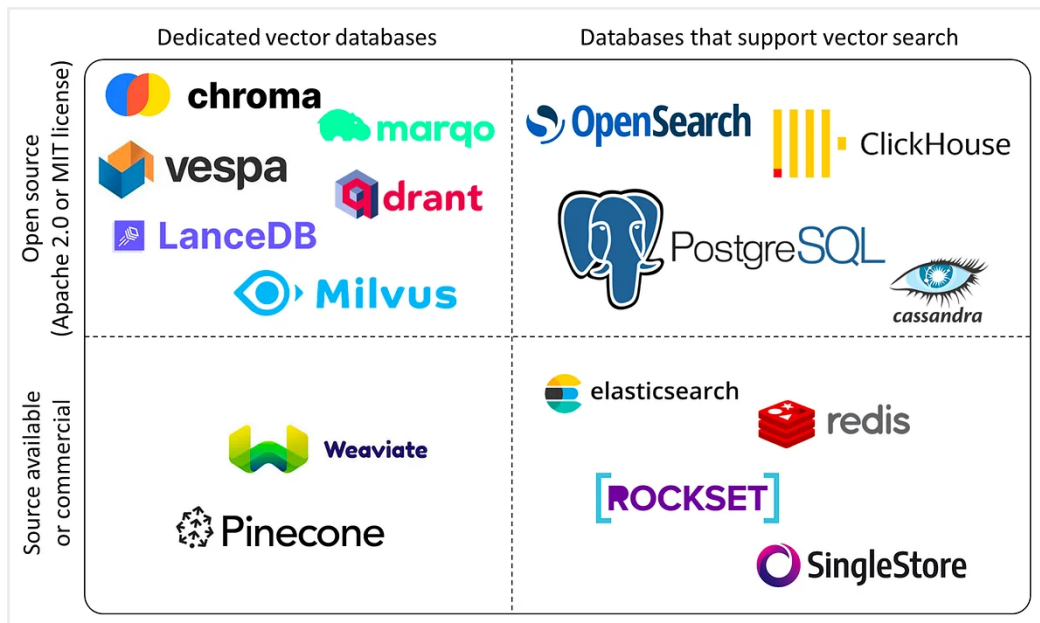
[Press enter or click to view image in full size](#)



Vector search

Vector search is the retrieval layer of a vector database used to discover and compare similar data points. Rather than matching exact keywords or values,

- **Cosine similarity:** Measures the angular distance between vectors to determine how aligned they are in direction.
- **Jaccard similarity:** Compares the overlap between two sets relative to their total elements



Flujo 1 — indexación (lo haces una vez, offline):

"Tomas los documentos, los divides en chunks de ~512 tokens con overlap, cada chunk lo conviertes en un vector con un modelo de embeddings, y los guardas en una base de datos vectorial."

Flujo 2 — consulta (en tiempo real, cada pregunta):

"La pregunta entra, primero verificas el cache de Redis. Si ya existe esa pregunta, devuelves la respuesta al instante. Si no, embedes la pregunta con el mismo modelo, haces una búsqueda ANN en el vector DB para sacar los top-k chunks más similares, inyectas esos chunks en el prompt junto con la pregunta, y se lo mandas al LLM. La respuesta la devuelves al usuario y la guardas en cache para la próxima vez."

Mejorar, Dataset de testing de preguntas, mirar como se hace la indexación, probar la similaridad de embeddings